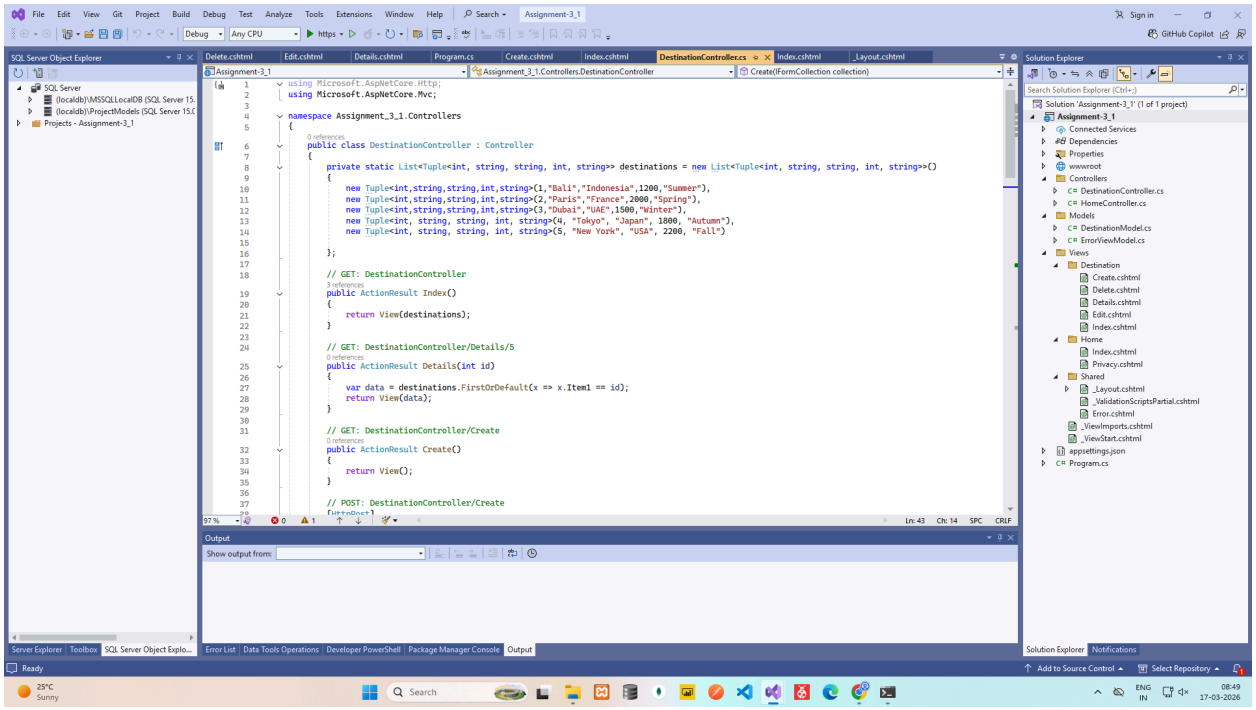
Assignment - 3

Assignment - 3 -> Q - 1



DestinationController.cs

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace Assignment_3_1.Controllers
{
    public class DestinationController : Controller
    {
        private static List<Tuple<int, string, string, int, string>> destinations = new List<Tuple<int, string, string, int, string>>()
        {
            new Tuple<int,string,string,int,string>(1,"Bali","Indonesia",1200,"Summer"),
            new Tuple<int,string,string,int,string>(2,"Paris","France",2000,"Spring"),
```

```
            new Tuple<int,string,string,int,string>(3,"Dubai","UAE",1500,"Winter"),
            new Tuple<int, string, string, int, string>(4, "Tokyo", "Japan", 1800, "Autumn"),
            new Tuple<int, string, string, int, string>(5, "New York", "USA", 2200, "Fall")

        };

        // GET: DestinationController
        public ActionResult Index()
        {
            return View(destinations);
        }

        // GET: DestinationController/Details/5
        public ActionResult Details(int id)
        {
            var data = destinations.FirstOrDefault(x => x.Item1 == id);
            return View(data);
        }

        // GET: DestinationController/Create
        public ActionResult Create()
        {
            return View();
        }

        // POST: DestinationController/Create
        [HttpPost]
        [ValidateAntiForgeryToken]
        public ActionResult Create(IFormCollection collection)
        {
            try
            {
                int id = Convert.ToInt32(collection["DestinationId"].ToString());
                string name = collection["DestinationName"].ToString() ?? "";
                string country = collection["Country"].ToString() ?? "";
                int cost = Convert.ToInt32(collection["EstimatedCost"].ToString());
                string season = collection["BestSeason"].ToString() ?? "";

                destinations.Add(new Tuple<int, string, string, int, string>(id, name, country, cost,
season));

                return RedirectToAction(nameof(Index));
            }
            catch
```

```
        {
            return View();
        }
    }

    // GET: DestinationController/Edit/5
    public ActionResult Edit(int id)
    {
        return View(destinations.SingleOrDefault(d => d.Item1 == id));
    }

    // POST: DestinationController/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public ActionResult Edit(int id, IFormCollection collection)
    {
        try
        {
            var dest = destinations.SingleOrDefault(d => d.Item1 == id);

            destinations.Remove(dest);

            string name = collection["DestinationName"].ToString() ?? "";
            string country = collection["Country"].ToString() ?? "";
            int cost = Convert.ToInt32(collection["EstimatedCost"].ToString());
            string season = collection["BestSeason"].ToString() ?? "";

            destinations.Add(new Tuple<int, string, string, int, string>(id, name, country, cost,
season));

            return RedirectToAction(nameof(Index));
        }
        catch
        {
            return View();
        }
    }

    // GET: DestinationController/Delete/5
    public ActionResult Delete(int id)
    {
        return View(destinations.SingleOrDefault(d => d.Item1 == id));
    }
```

```
        // POST: DestinationController/Delete/5
        [HttpPost]
        [ValidateAntiForgeryToken]
        public ActionResult Delete(int id, IFormCollection collection)
        {
            try
            {
                var dest = destinations.SingleOrDefault(d => d.Item1 == id);

                if (dest != null)
                    destinations.Remove(dest);

                return RedirectToAction(nameof(Index));
            }
            catch
            {
                return View();
            }
        }
    }
}
```

Create.cshtml

```
<h1>Create New Destination</h1>

<form method="post">
    <p>
        <label>Destination ID:</label><br />
        <input type="number" name="DestinationId" required />
    </p>
    <p>
        <label>Destination Name:</label><br />
        <input type="text" name="DestinationName" required />
    </p>
    <p>
        <label>Country:</label><br />
        <input type="text" name="Country" required />
    </p>
    <p>
        <label>Estimated Cost:</label><br />
        <input type="number" name="EstimatedCost" required />
```

```
    </p>
    <p>
        <label>Best Season:</label><br />
        <input type="text" name="BestSeason" required />
    </p>
    <p>
        <button type="submit">Save</button>
        <a href="/Destination">Cancel</a>
    </p>
</form>
```

Index.cshtml

```
@model List<Tuple<int, string, string, int, string>>

<h1>Destinations List</h1>

<p>
    <a href="/Destination/Create">Add New Destination</a>
</p>

<table border="1" cellpadding="5" cellspacing="0">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Country</th>
        <th>Estimated Cost</th>
        <th>Best Season</th>
        <th>Actions</th>
    </tr>
    @foreach (var d in Model)
    {
        <tr>
            <td>@d.Item1</td>
            <td>@d.Item2</td>
            <td>@d.Item3</td>
            <td>@d.Item4</td>
            <td>@d.Item5</td>
            <td>
                <a href="/Destination/Details/@d.Item1">Details</a> |
                <a href="/Destination/Edit/@d.Item1">Edit</a> |
                <a href="/Destination/Delete/@d.Item1">Delete</a>
            </td>
```

```
        </tr>
    }
</table>
```

Delete.cshtml

```
@model Tuple<int, string, string, int, string>

<h1>Delete Destination</h1>

<p>Are you sure you want to delete this destination?</p>

<table border="1" cellpadding="5">
    <tr>
        <th>ID</th>
        <td>@Model.Item1</td>
    </tr>
    <tr>
        <th>Name</th>
        <td>@Model.Item2</td>
    </tr>
    <tr>
        <th>Country</th>
        <td>@Model.Item3</td>
    </tr>
    <tr>
        <th>Estimated Cost</th>
        <td>@Model.Item4</td>
    </tr>
    <tr>
        <th>Best Season</th>
        <td>@Model.Item5</td>
    </tr>
</table>

<form method="post">
    <input type="hidden" name="id" value="@Model.Item1" />
    <button type="submit">Yes, Delete</button>
    <a href="/Destination">Cancel</a>
</form>
```

Details.cshtml

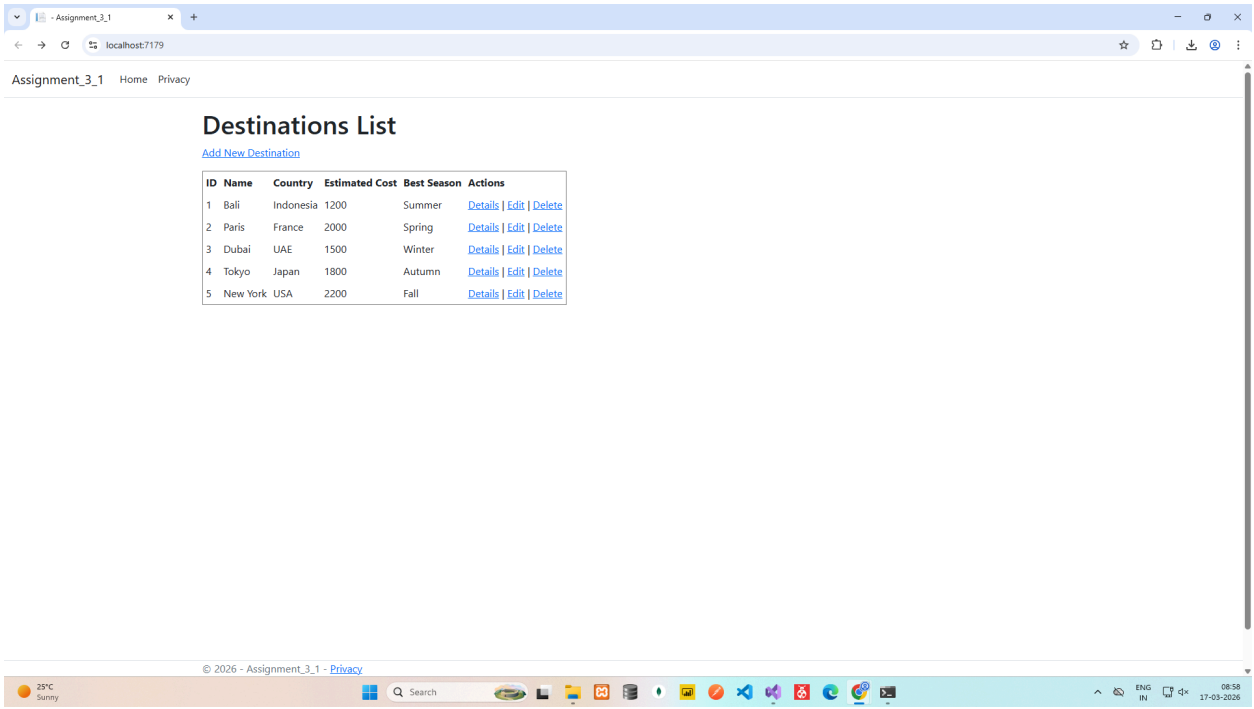
```
@model Tuple<int, string, string, int, string>

<h1>Destination Details</h1>

<table border="1" cellpadding="5">
    <tr>
        <th>ID</th>
        <td>@Model.Item1</td>
    </tr>
    <tr>
        <th>Name</th>
        <td>@Model.Item2</td>
    </tr>
    <tr>
        <th>Country</th>
        <td>@Model.Item3</td>
    </tr>
    <tr>
        <th>Estimated Cost</th>
        <td>@Model.Item4</td>
    </tr>
    <tr>
        <th>Best Season</th>
        <td>@Model.Item5</td>
    </tr>
</table>

<p>
    <a href="/Destination/Edit/@Model.Item1">Edit</a> |
    <a href="/Destination">Back to List</a>
</p>
```

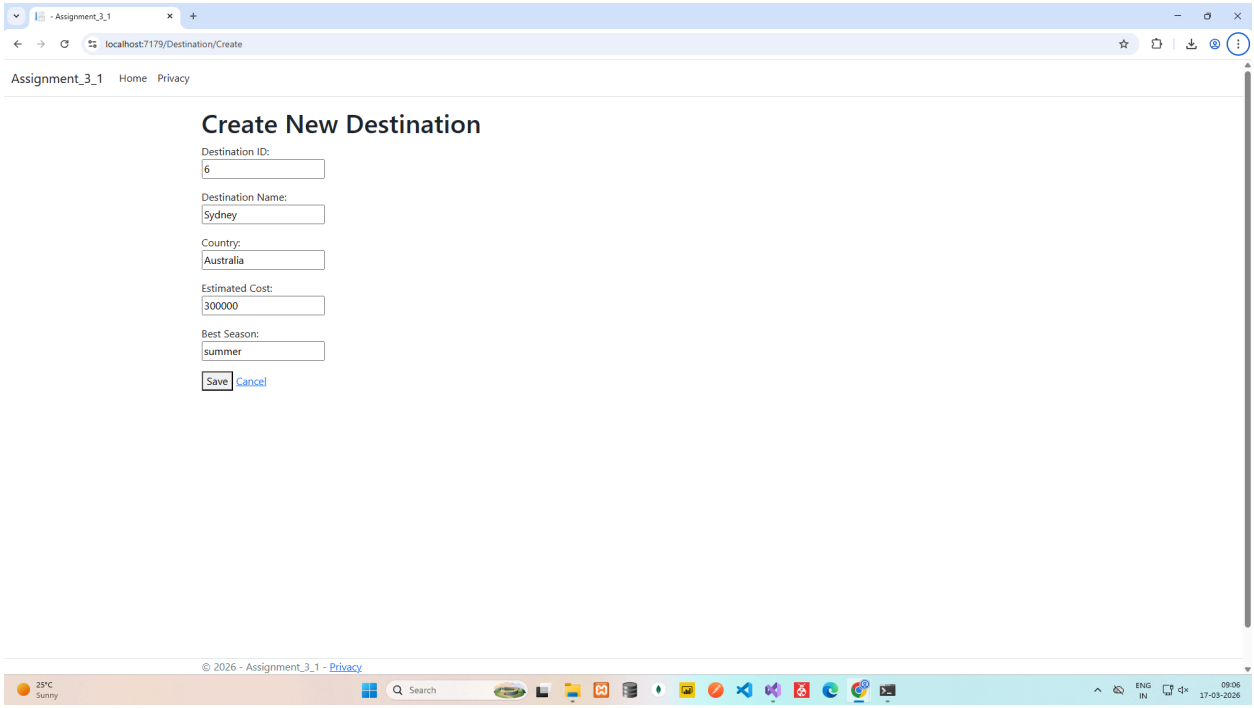
Edit.cshtml

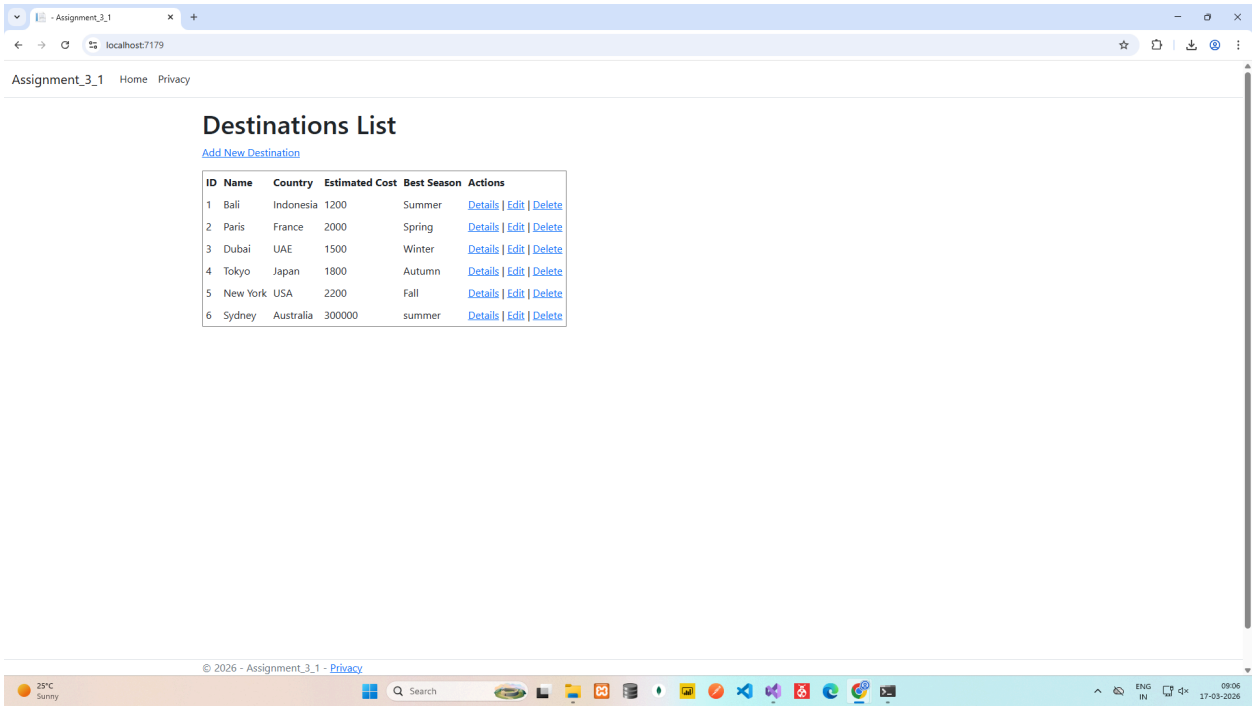
```
@model Tuple<int, string, string, int, string>

<h1>Edit Destination</h1>

<form method="post">
    <input type="hidden" name="id" value="@Model.Item1" />
```

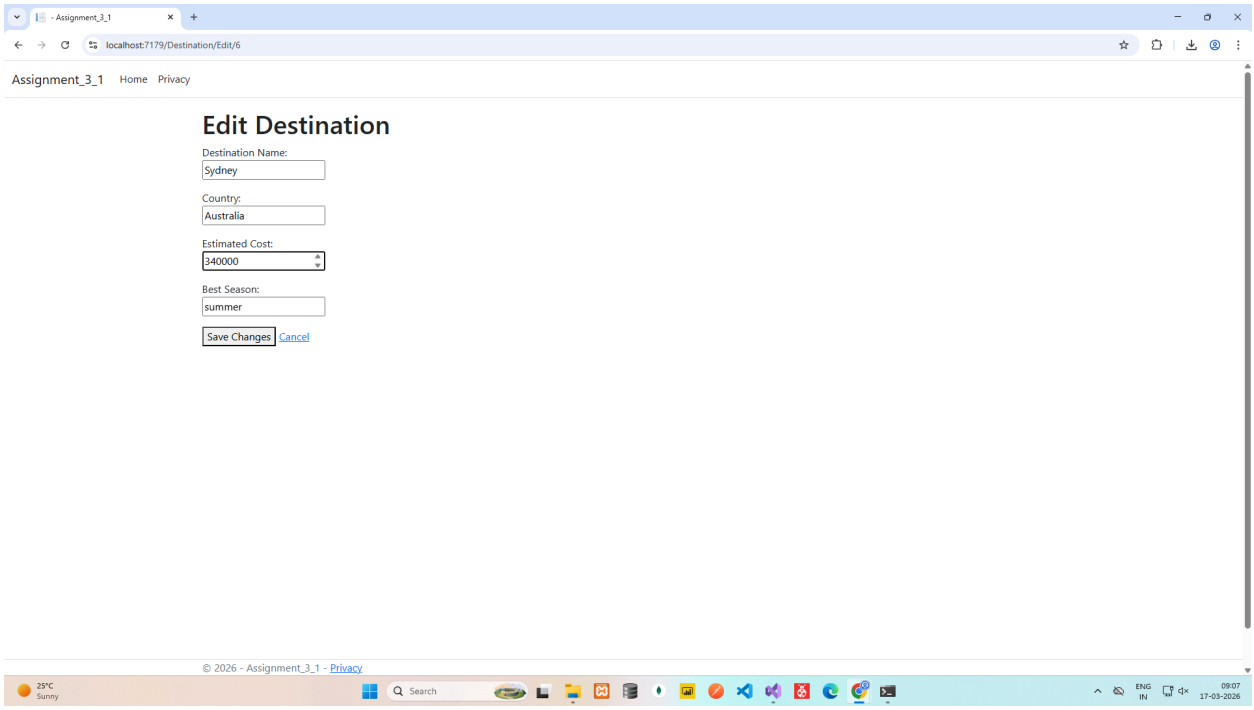
```
    <p>
        <label>Destination Name:</label><br />
        <input type="text" name="DestinationName" value="@Model.Item2" required />
    </p>
    <p>
        <label>Country:</label><br />
        <input type="text" name="Country" value="@Model.Item3" required />
    </p>
    <p>
        <label>Estimated Cost:</label><br />
        <input type="number" name="EstimatedCost" value="@Model.Item4" required />
    </p>
    <p>
        <label>Best Season:</label><br />
        <input type="text" name="BestSeason" value="@Model.Item5" required />
    </p>
    <p>
        <button type="submit">Save Changes</button>
        <a href="/Destination">Cancel</a>
    </p>
</form>
```

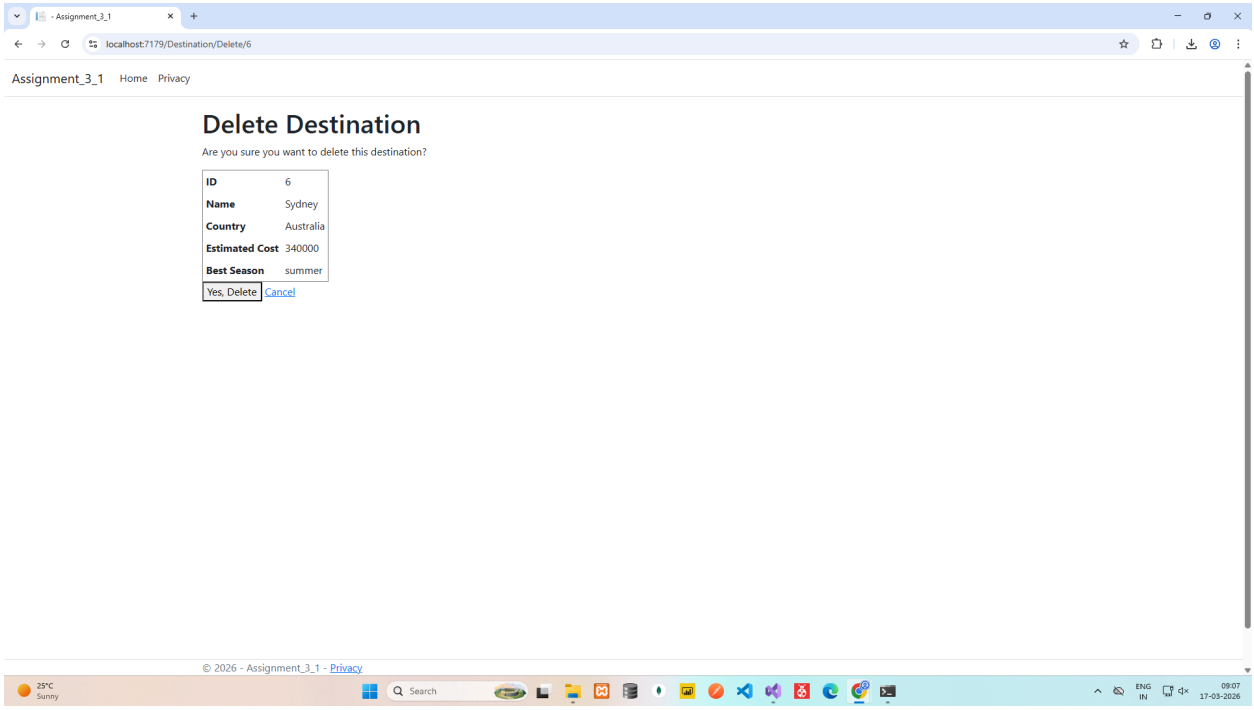
OUTPUT

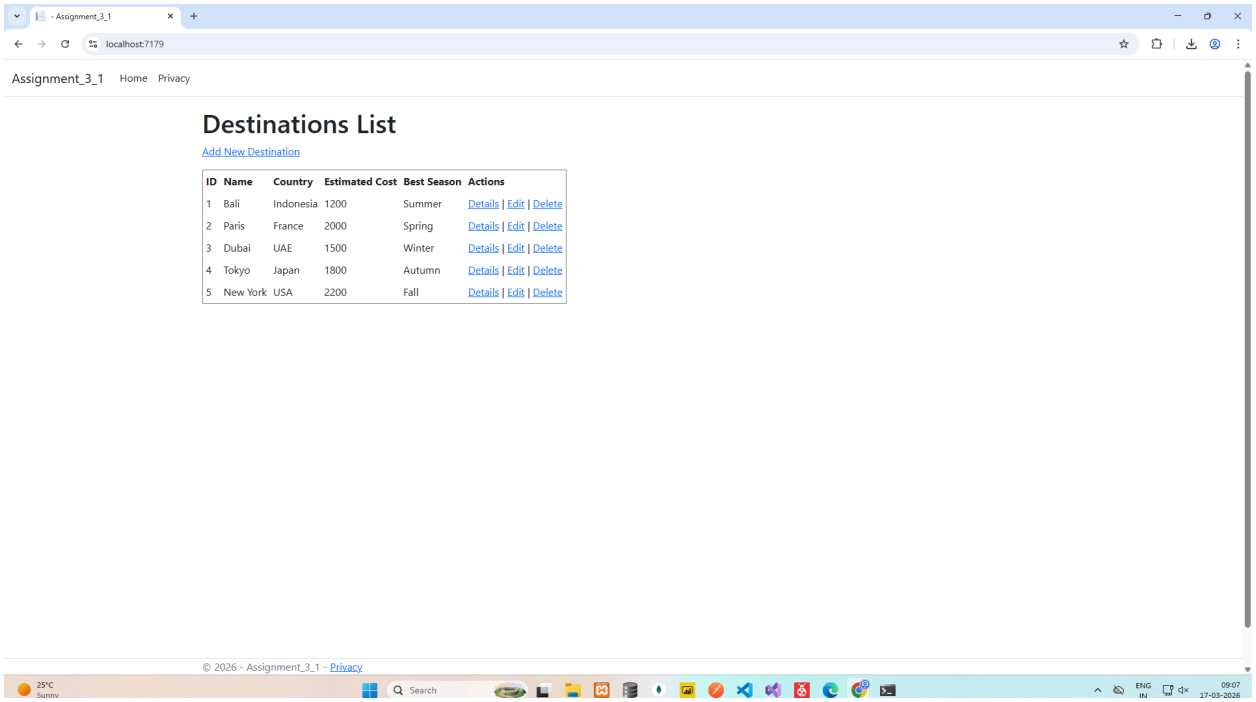



- Assignment_3_1
localhost:7179/Destination/Create
Assignment_3_1 Home Privacy
Create New Destination
Destination ID:
6
Destination Name:
Sydney
Country:
Australia
Estimated Cost:
300000
Best Season:
summer
Save Cancel
© 2026 - Assignment_3_1 - Privacy


- Assignment_3_1
localhost:7179
Assignment_3_1 Home Privacy
Destinations List
Add New Destination
ID Name Country Estimated Cost Best Season Actions
1 Bali Indonesia 1200 Summer Details | Edit | Delete
2 Paris France 2000 Spring Details | Edit | Delete
3 Dubai UAE 1500 Winter Details | Edit | Delete
4 Tokyo Japan 1800 Autumn Details | Edit | Delete
5 New York USA 2200 Fall Details | Edit | Delete
6 Sydney Australia 300000 summer Details | Edit | Delete
© 2026 - Assignment_3_1 - Privacy


- Assignment_3_1
localhost:7179/Destination/Edit/6
Assignment_3_1
Home
Privacy
Edit Destination
Destination Name:
Sydney
Country:
Australia
Estimated Cost:
340000
Best Season:
summer
Save Changes
Cancel
© 2026 - Assignment_3_1 - Privacy
25°C
Sunny
Search
ENG
IN
09:07
17-03-2026


- Assignment_3_1
localhost:7179/Destination/Delete/6
Assignment_3_1
Home
Privacy
Delete Destination
Are you sure you want to delete this destination?
ID 6
Name Sydney
Country Australia
Estimated Cost 340000
Best Season summer
Yes, Delete
Cancel
© 2026 - Assignment_3_1 - Privacy
25°C
Sunny
Search
ENG
IN
09:07
17-03-2026



Assignment - 3 -> Q - 2

VehiclesController.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Assignment_3_2.Models;

namespace Assignment_3_2.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class VehiclesController : ControllerBase
    {
        private readonly Ict2vehicleDbContext _context;
```

```
public VehiclesController(Ict2vehicleDbContext context)
{
    _context = context;
}

// GET: api/Vehicles
[HttpGet]
public async Task<ActionResult<IEnumerable<Vehicle>>> GetVehicles()
{
    return await _context.Vehicles.ToListAsync();
}

// GET: api/Vehicles/5
[HttpGet("{id}")]
public async Task<ActionResult<Vehicle>> GetVehicle(int id)
{
    var vehicle = await _context.Vehicles.FindAsync(id);

    if (vehicle == null)
    {
        return NotFound();
    }

    return vehicle;
}

// PUT: api/Vehicles/5
// To protect from overposting attacks, see https://go.microsoft.com/fwlink/?linkid=2123754
[HttpPut("{id}")]
public async Task<IActionResult> PutVehicle(int id, Vehicle vehicle)
{
    if (id != vehicle.VehicleId)
    {
        return BadRequest();
    }

    _context.Entry(vehicle).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
```

```
            if (!VehicleExists(id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }

        return NoContent();
    }

    // POST: api/Vehicles
    // To protect from overposting attacks, see https://go.microsoft.com/fwlink/?linkid=2123754
    [HttpPost]
    public async Task<ActionResult<Vehicle>> PostVehicle(Vehicle vehicle)
    {
        _context.Vehicles.Add(vehicle);
        await _context.SaveChangesAsync();

        return CreatedAtAction("GetVehicle", new { id = vehicle.VehicleId }, vehicle);
    }

    // DELETE: api/Vehicles/5
    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteVehicle(int id)
    {
        var vehicle = await _context.Vehicles.FindAsync(id);
        if (vehicle == null)
        {
            return NotFound();
        }

        _context.Vehicles.Remove(vehicle);
        await _context.SaveChangesAsync();

        return NoContent();
    }

    private bool VehicleExists(int id)
    {
        return _context.Vehicles.Any(e => e.VehicleId == id);
    }
```

```
    }
}
```

Ict2vehicleDbContext.cs

```
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;

namespace Assignment_3_2.Models;

public partial class Ict2vehicleDbContext : DbContext
{
    public Ict2vehicleDbContext()
    {
    }

    public Ict2vehicleDbContext(DbContextOptions<Ict2vehicleDbContext> options)
        : base(options)
    {
    }

    public virtual DbSet<Vehicle> Vehicles { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
#warning To protect potentially sensitive information in your connection string, you should move it out of source code. You can avoid scaffolding the connection string by using the Name= syntax to read it from configuration - see https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance on storing connection strings, see https://go.microsoft.com/fwlink/?LinkId=723263.
        => optionsBuilder.UseSqlServer("Data Source=DESKTOP-E6KULOA;Initial Catalog=ICT2VehicleDB;Integrated Security=True;Encrypt=True;Trust Server Certificate=True");

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Vehicle>(entity =>
        {
            entity.HasKey(e => e.VehicleId).HasName("PK__Vehicles__476B54920679FB30");

            entity.Property(e => e.Color).HasMaxLength(50);
            entity.Property(e => e.Price).HasColumnType("decimal(18, 0)");
        });

        OnModelCreatingPartial(modelBuilder);
```

```
    }

    partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}
```

Vehicle.cs

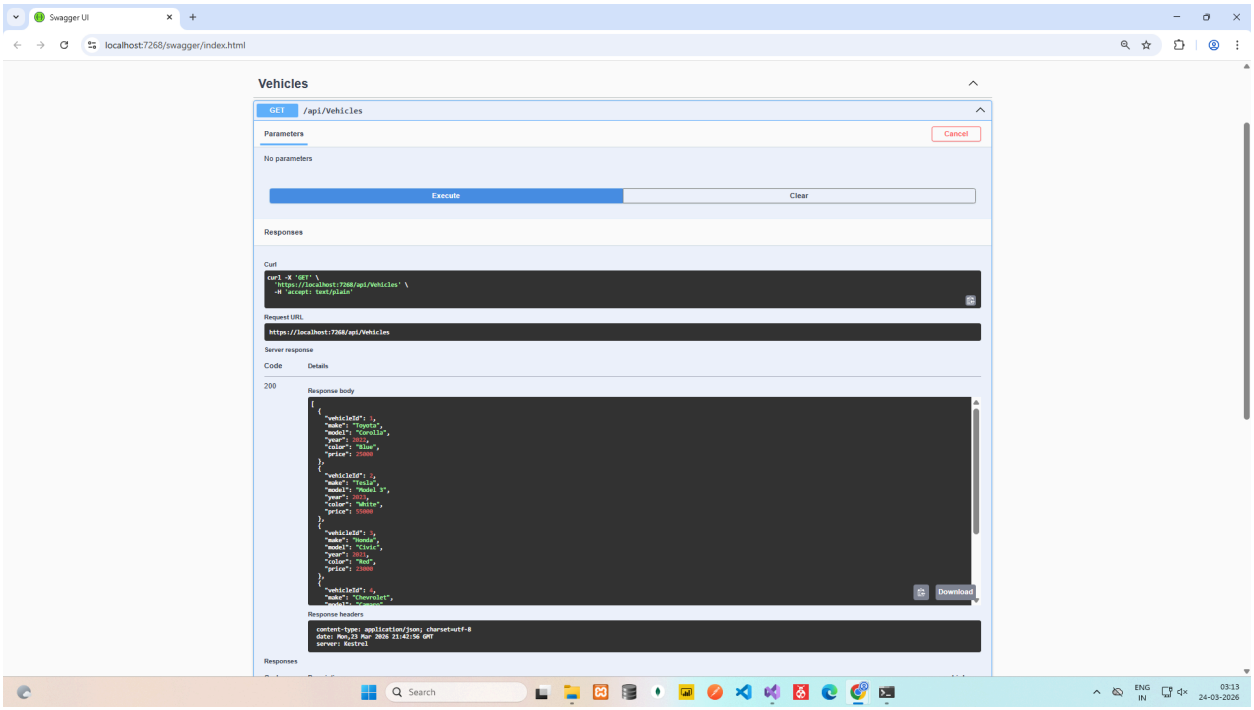
```
using System;
using System.Collections.Generic;

namespace Assignment_3_2.Models;

public partial class Vehicle
{
    public int VehicleId { get; set; }

    public string? Make { get; set; }

    public string? Model { get; set; }

    public int? Year { get; set; }

    public string? Color { get; set; }

    public decimal? Price { get; set; }
}
```

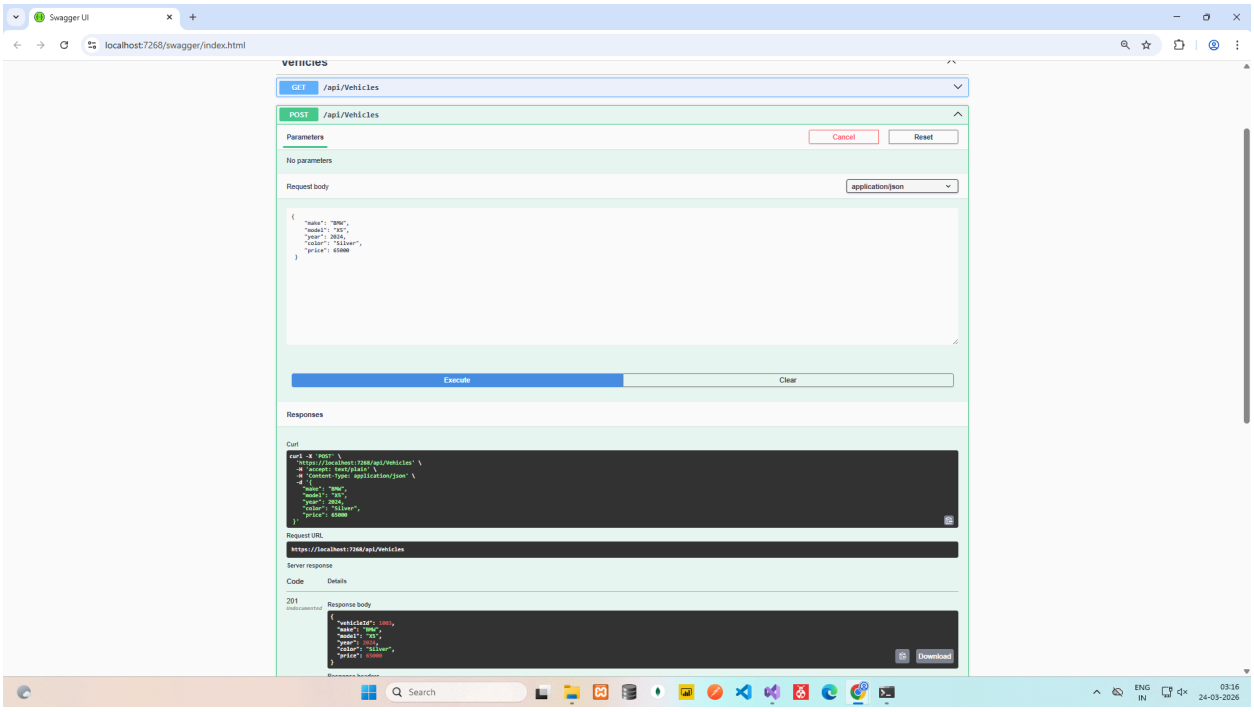
Program.cs

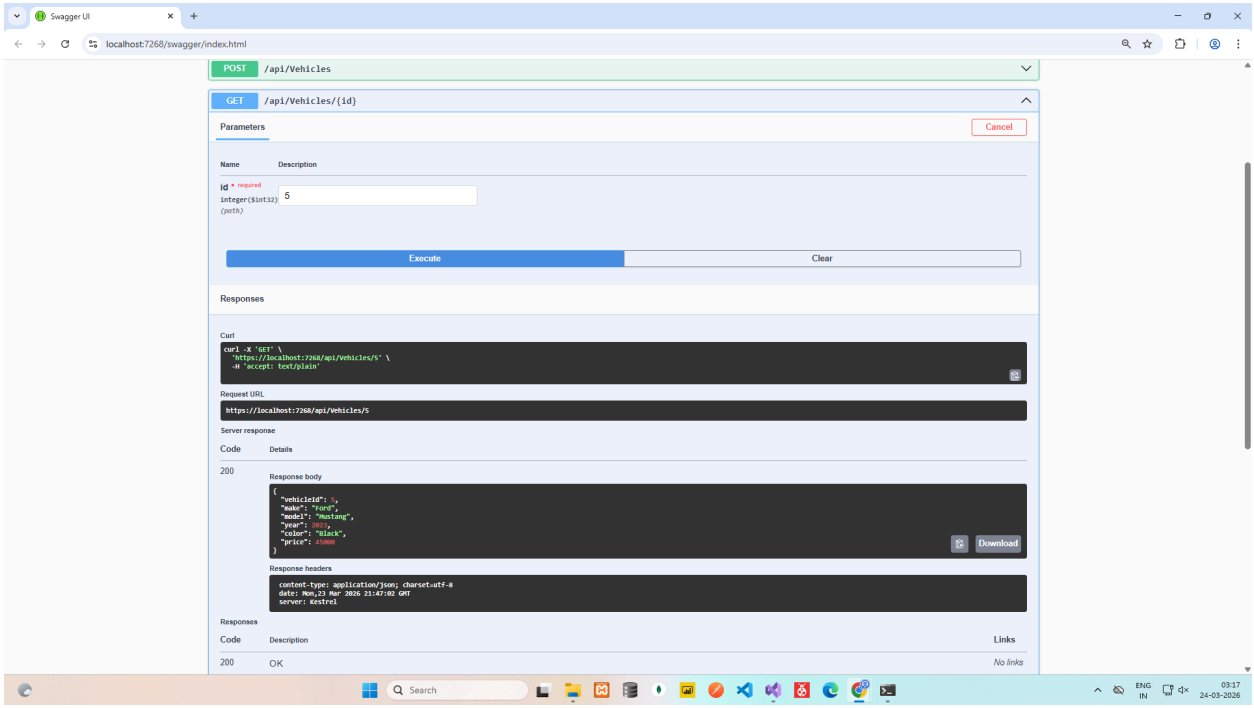
```
using Assignment_3_2.Models;
using Microsoft.EntityFrameworkCore;

namespace Assignment_3_2
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);
```

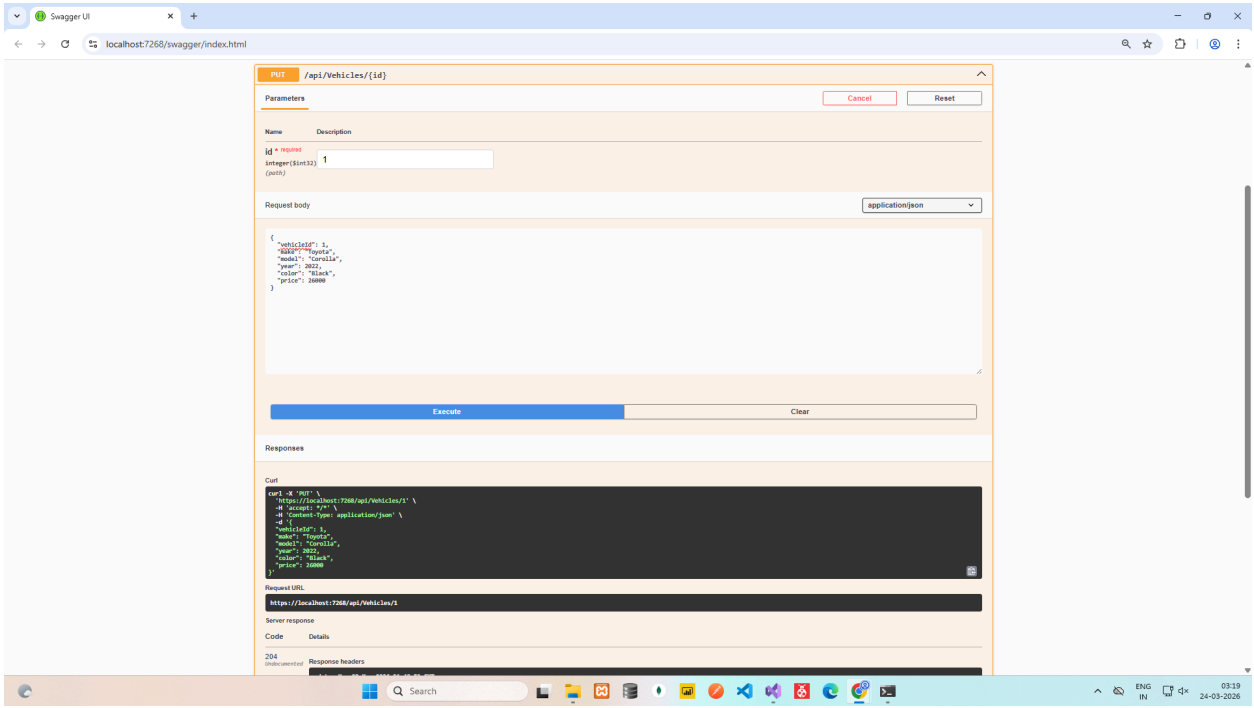
```
        // Add services to the container.

        builder.Services.AddControllers();
        // Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
        builder.Services.AddEndpointsApiExplorer();
        builder.Services.AddSwaggerGen();
        builder.Services.AddDbContext<Ict2vehicleDbContext>(
           option => option.UseSqlServer(
              builder.Configuration.GetConnectionString("Vehicleconstring")));

        var app = builder.Build();

        // Configure the HTTP request pipeline.
        if (app.Environment.IsDevelopment())
        {
           app.UseSwagger();
           app.UseSwaggerUI();
        }

        app.UseHttpsRedirection();

        app.UseAuthorization();


        app.MapControllers();

        app.Run();
     }
  }
}
```

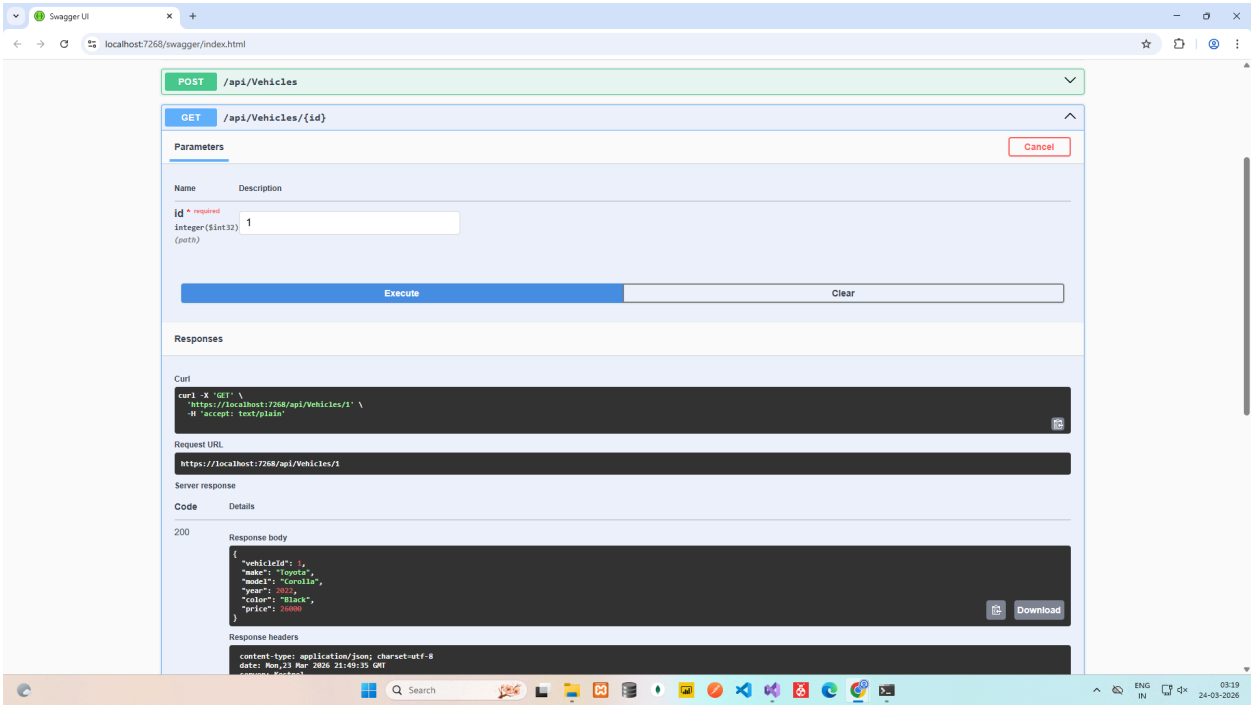
OUTPUT

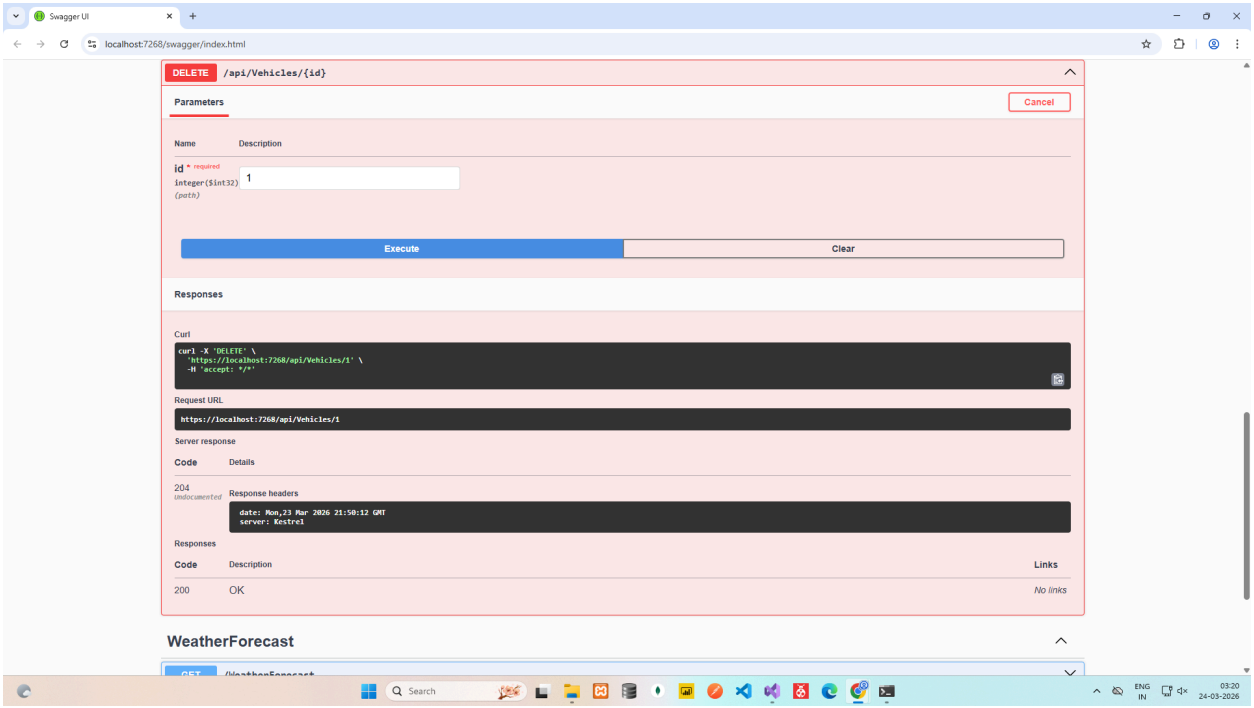

Swagger UI
localhost:7268/swagger/index.html
Vehicles
GET /api/Vehicles
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://localhost:7268/api/Vehicles' \
-H 'accept: text/plain'
Request URL
https://localhost:7268/api/Vehicles
Server response
Code
Details
200
Response body
Download
Response headers
content-type: application/json; charset=utf-8
date: Mon,23 Mar 2026 21:42:56 GMT
server: Kestrel
Responses
Search
ENG
IN
03:13
24-03-2026

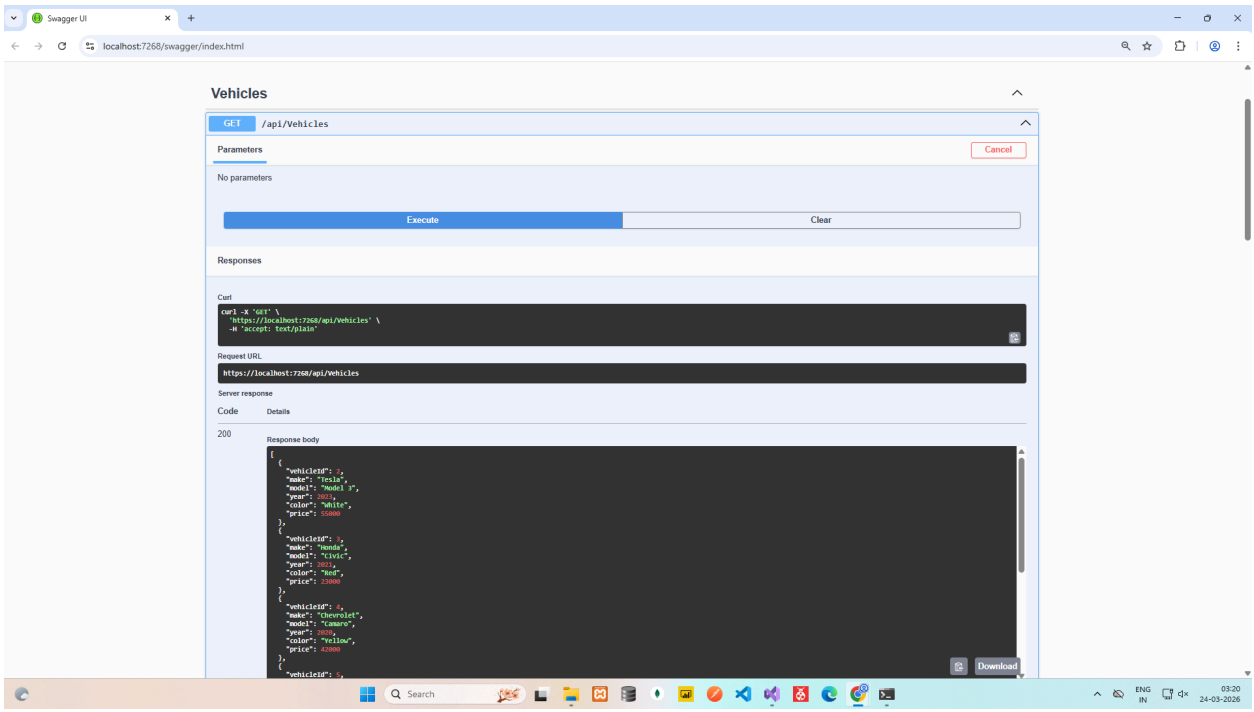

Swagger UI
localhost:7268/swagger/index.html
Vehicles
GET /api/Vehicles
POST /api/Vehicles
Parameters
Cancel
Reset
No parameters
Request body
application/json
Execute
Clear
Responses
Curl
Request URL
https://localhost:7268/api/Vehicles
Server response
Code
Details
201
Response body
Download
Search
ENG
IN
03:16
24-03-2026


Swagger UI
localhost:7268/swagger/index.html
POST /api/Vehicles
GET /api/Vehicles/{id}
Parameters
Cancel
Name
Description
id * required
integer($int32)
(path)
5
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://localhost:7268/api/Vehicles/5' \
-H 'accept: text/plain'
Request URL
https://localhost:7268/api/Vehicles/5
Server response
Code
Details
200
Response body
{
"vehicleId": 5,
"make": "Ford",
"model": "Mustang",
"year": 2023,
"color": "Black",
"price": 45000
}
Download
Response headers
content-type: application/json; charset=utf-8
date: Mon,23 Mar 2026 21:47:02 GMT
server: Kestrel
Responses
Code
Description
Links
200
OK
No links
Search
ENG
IN
03:17
24-03-2026


Swagger UI
localhost:7268/swagger/index.html
PUT /api/Vehicles/{id}
Parameters
Cancel
Reset
Name
Description
id * required
integer($int32)
(path)
1
Request body
application/json
{
"vehicleId": 1,
"make": "Toyota",
"model": "Corolla",
"year": 2022,
"color": "Black",
"price": 26000
}
Execute
Clear
Responses
Curl
curl -X 'PUT' \
'https://localhost:7268/api/Vehicles/1' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"vehicleId": 1,
"make": "Toyota",
"model": "Corolla",
"year": 2022,
"color": "Black",
"price": 26000
}'
Request URL
https://localhost:7268/api/Vehicles/1
Server response
Code
Details
204
Response headers
Search
ENG
IN
03:19
24-03-2026


Swagger UI
localhost:7268/swagger/index.html
POST /api/Vehicles
GET /api/Vehicles/{id}
Parameters
Cancel
Name
Description
id * required
integer($int32)
1
(path)
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://localhost:7268/api/Vehicles/1' \
-H 'accept: text/plain'
Request URL
https://localhost:7268/api/Vehicles/1
Server response
Code
Details
200
Response body
{
"vehicleId": 1,
"make": "Toyota",
"model": "Corolla",
"year": 2023,
"color": "Black",
"price": 25000
}
Download
Response headers
content-type: application/json; charset=utf-8
date: Mon,23 Mar 2026 21:49:35 GMT
Search
ENG
IN
03:19
24-03-2026


Swagger UI
localhost:7268/swagger/index.html
DELETE /api/Vehicles/{id}
Parameters
Cancel
Name
Description
id * required
integer($int32)
1
(path)
Execute
Clear
Responses
Curl
curl -X 'DELETE' \
'https://localhost:7268/api/Vehicles/1' \
-H 'accept: */*'
Request URL
https://localhost:7268/api/Vehicles/1
Server response
Code
Details
204
Response headers
date: Mon,23 Mar 2026 21:50:12 GMT
server: Kestrel
Responses
Code
Description
Links
200
OK
No links
WeatherForecast
Search
ENG
IN
03:20
24-03-2026



Assignment - 3 -> Q - 3

CreateIdentitySchema.cs

```
using System;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Migrations;

namespace Assignment_3_3.Data.Migrations
{
    public partial class CreateIdentitySchema : Migration
    {
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "AspNetRoles",
                columns: table => new
```

```
            {
                Id = table.Column<string>(nullable: false),
                Name = table.Column<string>(maxLength: 256, nullable: true),
                NormalizedName = table.Column<string>(maxLength: 256, nullable: true),
                ConcurrencyStamp = table.Column<string>(nullable: true)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_AspNetRoles", x => x.Id);
            });

        migrationBuilder.CreateTable(
            name: "AspNetUsers",
            columns: table => new
            {
                Id = table.Column<string>(nullable: false),
                UserName = table.Column<string>(maxLength: 256, nullable: true),
                NormalizedUserName = table.Column<string>(maxLength: 256, nullable: true),
                Email = table.Column<string>(maxLength: 256, nullable: true),
                NormalizedEmail = table.Column<string>(maxLength: 256, nullable: true),
                EmailConfirmed = table.Column<bool>(nullable: false),
                PasswordHash = table.Column<string>(nullable: true),
                SecurityStamp = table.Column<string>(nullable: true),
                ConcurrencyStamp = table.Column<string>(nullable: true),
                PhoneNumber = table.Column<string>(nullable: true),
                PhoneNumberConfirmed = table.Column<bool>(nullable: false),
                TwoFactorEnabled = table.Column<bool>(nullable: false),
                LockoutEnd = table.Column<DateTimeOffset>(nullable: true),
                LockoutEnabled = table.Column<bool>(nullable: false),
                AccessFailedCount = table.Column<int>(nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_AspNetUsers", x => x.Id);
            });

        migrationBuilder.CreateTable(
            name: "AspNetRoleClaims",
            columns: table => new
            {
                Id = table.Column<int>(nullable: false)
                    .Annotation("SqlServer:ValueGenerationStrategy",
SqlServerValueGenerationStrategy.IdentityColumn),
                RoleId = table.Column<string>(nullable: false),
```

```
                ClaimType = table.Column<string>(nullable: true),
                ClaimValue = table.Column<string>(nullable: true)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_AspNetRoleClaims", x => x.Id);
                table.ForeignKey(
                    name: "FK_AspNetRoleClaims_AspNetRoles_RoleId",
                    column: x => x.RoleId,
                    principalTable: "AspNetRoles",
                    principalColumn: "Id",
                    onDelete: ReferentialAction.Cascade);
            });

        migrationBuilder.CreateTable(
            name: "AspNetUserClaims",
            columns: table => new
            {
                Id = table.Column<int>(nullable: false)
                    .Annotation("SqlServer:ValueGenerationStrategy",
SqlServerValueGenerationStrategy.IdentityColumn),
                UserId = table.Column<string>(nullable: false),
                ClaimType = table.Column<string>(nullable: true),
                ClaimValue = table.Column<string>(nullable: true)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_AspNetUserClaims", x => x.Id);
                table.ForeignKey(
                    name: "FK_AspNetUserClaims_AspNetUsers_UserId",
                    column: x => x.UserId,
                    principalTable: "AspNetUsers",
                    principalColumn: "Id",
                    onDelete: ReferentialAction.Cascade);
            });

        migrationBuilder.CreateTable(
            name: "AspNetUserLogins",
            columns: table => new
            {
                LoginProvider = table.Column<string>(maxLength: 128, nullable: false),
                ProviderKey = table.Column<string>(maxLength: 128, nullable: false),
                ProviderDisplayName = table.Column<string>(nullable: true),
                UserId = table.Column<string>(nullable: false)
```

```
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_AspNetUserLogins", x => new { x.LoginProvider,
x.ProviderKey });
                table.ForeignKey(
                    name: "FK_AspNetUserLogins_AspNetUsers_UserId",
                    column: x => x.UserId,
                    principalTable: "AspNetUsers",
                    principalColumn: "Id",
                    onDelete: ReferentialAction.Cascade);
            });

        migrationBuilder.CreateTable(
            name: "AspNetUserRoles",
            columns: table => new
            {
                UserId = table.Column<string>(nullable: false),
                RoleId = table.Column<string>(nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_AspNetUserRoles", x => new { x.UserId, x.RoleId });
                table.ForeignKey(
                    name: "FK_AspNetUserRoles_AspNetRoles_RoleId",
                    column: x => x.RoleId,
                    principalTable: "AspNetRoles",
                    principalColumn: "Id",
                    onDelete: ReferentialAction.Cascade);
                table.ForeignKey(
                    name: "FK_AspNetUserRoles_AspNetUsers_UserId",
                    column: x => x.UserId,
                    principalTable: "AspNetUsers",
                    principalColumn: "Id",
                    onDelete: ReferentialAction.Cascade);
            });

        migrationBuilder.CreateTable(
            name: "AspNetUserTokens",
            columns: table => new
            {
                UserId = table.Column<string>(nullable: false),
                LoginProvider = table.Column<string>(maxLength: 128, nullable: false),
                Name = table.Column<string>(maxLength: 128, nullable: false),
```

```
                Value = table.Column<string>(nullable: true)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_AspNetUserTokens", x => new { x.UserId, x.LoginProvider,
x.Name });
                table.ForeignKey(
                    name: "FK_AspNetUserTokens_AspNetUsers_UserId",
                    column: x => x.UserId,
                    principalTable: "AspNetUsers",
                    principalColumn: "Id",
                    onDelete: ReferentialAction.Cascade);
            });

        migrationBuilder.CreateIndex(
            name: "IX_AspNetRoleClaims_RoleId",
            table: "AspNetRoleClaims",
            column: "RoleId");

        migrationBuilder.CreateIndex(
            name: "RoleNameIndex",
            table: "AspNetRoles",
            column: "NormalizedName",
            unique: true,
            filter: "[NormalizedName] IS NOT NULL");

        migrationBuilder.CreateIndex(
            name: "IX_AspNetUserClaims_UserId",
            table: "AspNetUserClaims",
            column: "UserId");

        migrationBuilder.CreateIndex(
            name: "IX_AspNetUserLogins_UserId",
            table: "AspNetUserLogins",
            column: "UserId");

        migrationBuilder.CreateIndex(
            name: "IX_AspNetUserRoles_RoleId",
            table: "AspNetUserRoles",
            column: "RoleId");

        migrationBuilder.CreateIndex(
            name: "EmailIndex",
            table: "AspNetUsers",
```

```
            column: "NormalizedEmail");

        migrationBuilder.CreateIndex(
            name: "UserNameIndex",
            table: "AspNetUsers",
            column: "NormalizedUserName",
            unique: true,
            filter: "[NormalizedUserName] IS NOT NULL");
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(
            name: "AspNetRoleClaims");

        migrationBuilder.DropTable(
            name: "AspNetUserClaims");

        migrationBuilder.DropTable(
            name: "AspNetUserLogins");

        migrationBuilder.DropTable(
            name: "AspNetUserRoles");

        migrationBuilder.DropTable(
            name: "AspNetUserTokens");

        migrationBuilder.DropTable(
            name: "AspNetRoles");

        migrationBuilder.DropTable(
            name: "AspNetUsers");
    }
  }
}
```

ApplicationDbContextModelSnapshot.cs

```
// <auto-generated />
using System;
using Assignment_3_3.Data;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Infrastructure;
```

```
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;

#nullable disable

namespace Assignment_3_3.Data.Migrations
{
    [DbContext(typeof(ApplicationDbContext))]
    partial class ApplicationDbContextModelSnapshot : ModelSnapshot
    {
        protected override void BuildModel(ModelBuilder modelBuilder)
        {
#pragma warning disable 612, 618
            modelBuilder
                .HasAnnotation("ProductVersion", "8.0.23")
                .HasAnnotation("Relational:MaxIdentifierLength", 128);

            SqlServerModelBuilderExtensions.UseIdentityColumns(modelBuilder);

            modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityRole", b =>
                {
                    b.Property<string>("Id")
                        .HasColumnType("nvarchar(450)");

                    b.Property<string>("ConcurrencyStamp")
                        .IsConcurrencyToken()
                        .HasColumnType("nvarchar(max)");

                    b.Property<string>("Name")
                        .HasMaxLength(256)
                        .HasColumnType("nvarchar(256)");

                    b.Property<string>("NormalizedName")
                        .HasMaxLength(256)
                        .HasColumnType("nvarchar(256)");

                    b.HasKey("Id");

                    b.HasIndex("NormalizedName")
                        .IsUnique()
                        .HasDatabaseName("RoleNameIndex")
                        .HasFilter("[NormalizedName] IS NOT NULL");

                    b.ToTable("AspNetRoles", (string)null);
```

```
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityRoleClaim<string>", b =>
    {
        b.Property<int>("Id")
            .ValueGeneratedOnAdd()
            .HasColumnType("int");

        SqlServerPropertyBuilderExtensions.UseIdentityColumn(b.Property<int>("Id"));

        b.Property<string>("ClaimType")
            .HasColumnType("nvarchar(max)");

        b.Property<string>("ClaimValue")
            .HasColumnType("nvarchar(max)");

        b.Property<string>("RoleId")
            .IsRequired()
            .HasColumnType("nvarchar(450)");

        b.HasKey("Id");

        b.HasIndex("RoleId");

        b.ToTable("AspNetRoleClaims", (string)null);
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUser", b =>
    {
        b.Property<string>("Id")
            .HasColumnType("nvarchar(450)");

        b.Property<int>("AccessFailedCount")
            .HasColumnType("int");

        b.Property<string>("ConcurrencyStamp")
            .IsConcurrencyToken()
            .HasColumnType("nvarchar(max)");

        b.Property<string>("Email")
            .HasMaxLength(256)
            .HasColumnType("nvarchar(256)");

        b.Property<bool>("EmailConfirmed")
```

```
        .HasColumnType("bit");

    b.Property<bool>("LockoutEnabled")
        .HasColumnType("bit");

    b.Property<DateTimeOffset?>("LockoutEnd")
        .HasColumnType("datetimeoffset");

    b.Property<string>("NormalizedEmail")
        .HasMaxLength(256)
        .HasColumnType("nvarchar(256)");

    b.Property<string>("NormalizedUserName")
        .HasMaxLength(256)
        .HasColumnType("nvarchar(256)");

    b.Property<string>("PasswordHash")
        .HasColumnType("nvarchar(max)");

    b.Property<string>("PhoneNumber")
        .HasColumnType("nvarchar(max)");

    b.Property<bool>("PhoneNumberConfirmed")
        .HasColumnType("bit");

    b.Property<string>("SecurityStamp")
        .HasColumnType("nvarchar(max)");

    b.Property<bool>("TwoFactorEnabled")
        .HasColumnType("bit");

    b.Property<string>("UserName")
        .HasMaxLength(256)
        .HasColumnType("nvarchar(256)");

    b.HasKey("Id");

    b.HasIndex("NormalizedEmail")
        .HasDatabaseName("EmailIndex");

    b.HasIndex("NormalizedUserName")
        .IsUnique()
        .HasDatabaseName("UserNameIndex")
        .HasFilter("[NormalizedUserName] IS NOT NULL");
```

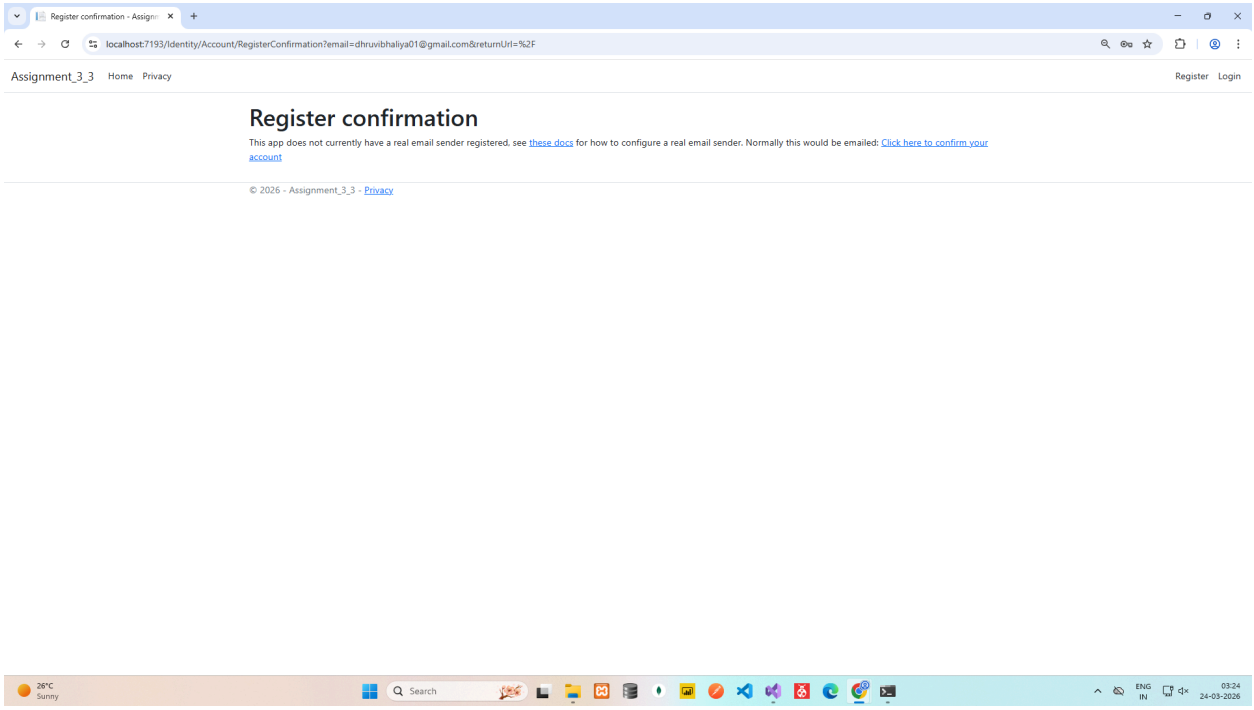
```
        b.ToTable("AspNetUsers", (string)null);
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUserClaim<string>", b =>
    {
        b.Property<int>("Id")
            .ValueGeneratedOnAdd()
            .HasColumnType("int");

        SqlServerPropertyBuilderExtensions.UseIdentityColumn(b.Property<int>("Id"));

        b.Property<string>("ClaimType")
            .HasColumnType("nvarchar(max)");

        b.Property<string>("ClaimValue")
            .HasColumnType("nvarchar(max)");

        b.Property<string>("UserId")
            .IsRequired()
            .HasColumnType("nvarchar(450)");

        b.HasKey("Id");

        b.HasIndex("UserId");

        b.ToTable("AspNetUserClaims", (string)null);
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUserLogin<string>", b =>
    {
        b.Property<string>("LoginProvider")
            .HasMaxLength(128)
            .HasColumnType("nvarchar(128)");

        b.Property<string>("ProviderKey")
            .HasMaxLength(128)
            .HasColumnType("nvarchar(128)");

        b.Property<string>("ProviderDisplayName")
            .HasColumnType("nvarchar(max)");

        b.Property<string>("UserId")
            .IsRequired()
```

```
            .HasColumnType("nvarchar(450)");

        b.HasKey("LoginProvider", "ProviderKey");

        b.HasIndex("UserId");

        b.ToTable("AspNetUserLogins", (string)null);
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUserRole<string>", b =>
    {
        b.Property<string>("UserId")
            .HasColumnType("nvarchar(450)");

        b.Property<string>("RoleId")
            .HasColumnType("nvarchar(450)");

        b.HasKey("UserId", "RoleId");

        b.HasIndex("RoleId");

        b.ToTable("AspNetUserRoles", (string)null);
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUserToken<string>", b =>
    {
        b.Property<string>("UserId")
            .HasColumnType("nvarchar(450)");

        b.Property<string>("LoginProvider")
            .HasMaxLength(128)
            .HasColumnType("nvarchar(128)");

        b.Property<string>("Name")
            .HasMaxLength(128)
            .HasColumnType("nvarchar(128)");

        b.Property<string>("Value")
            .HasColumnType("nvarchar(max)");

        b.HasKey("UserId", "LoginProvider", "Name");

        b.ToTable("AspNetUserTokens", (string)null);
    });
```

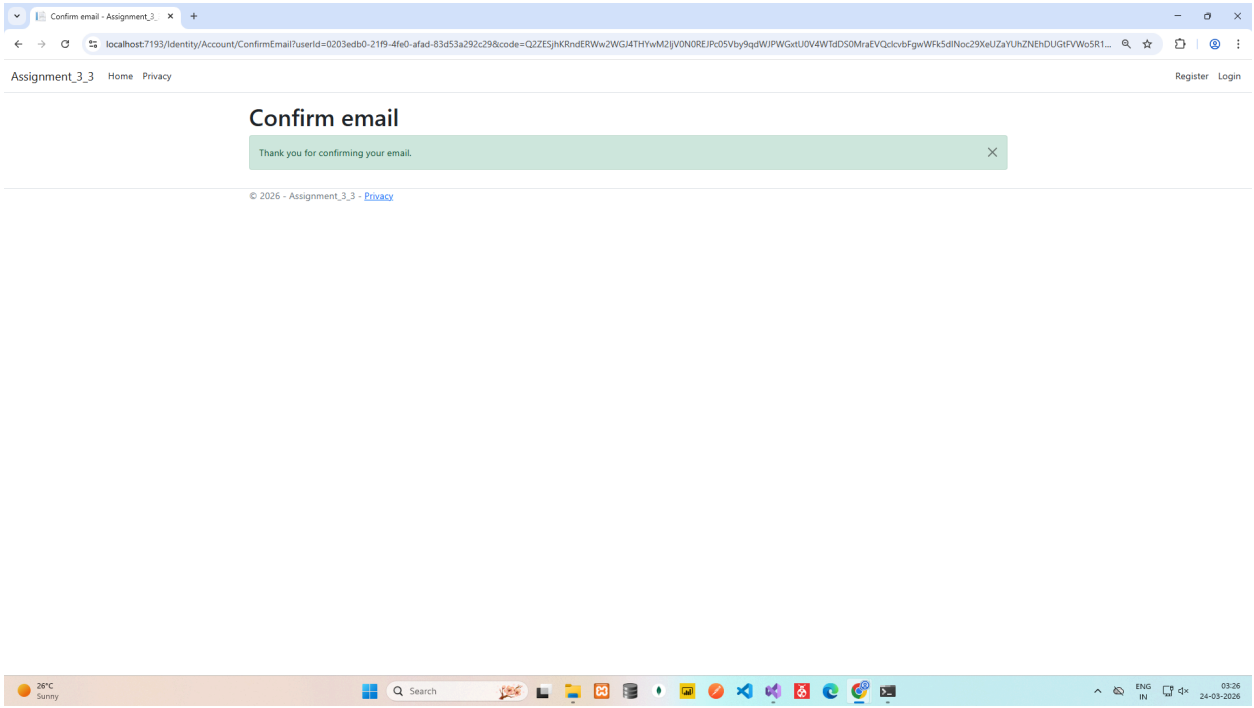
```
modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityRoleClaim<string>", b =>
    {
        b.HasOne("Microsoft.AspNetCore.Identity.IdentityRole", null)
            .WithMany()
            .HasForeignKey("RoleId")
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUserClaim<string>", b =>
    {
        b.HasOne("Microsoft.AspNetCore.Identity.IdentityUser", null)
            .WithMany()
            .HasForeignKey("UserId")
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUserLogin<string>", b =>
    {
        b.HasOne("Microsoft.AspNetCore.Identity.IdentityUser", null)
            .WithMany()
            .HasForeignKey("UserId")
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUserRole<string>", b =>
    {
        b.HasOne("Microsoft.AspNetCore.Identity.IdentityRole", null)
            .WithMany()
            .HasForeignKey("RoleId")
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();

        b.HasOne("Microsoft.AspNetCore.Identity.IdentityUser", null)
            .WithMany()
            .HasForeignKey("UserId")
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

modelBuilder.Entity("Microsoft.AspNetCore.Identity.IdentityUserToken<string>", b =>
```

```
            {
                b.HasOne("Microsoft.AspNetCore.Identity.IdentityUser", null)
                    .WithMany()
                    .HasForeignKey("UserId")
                    .OnDelete(DeleteBehavior.Cascade)
                    .IsRequired();
            });
#pragma warning restore 612, 618
        }
    }
}
```

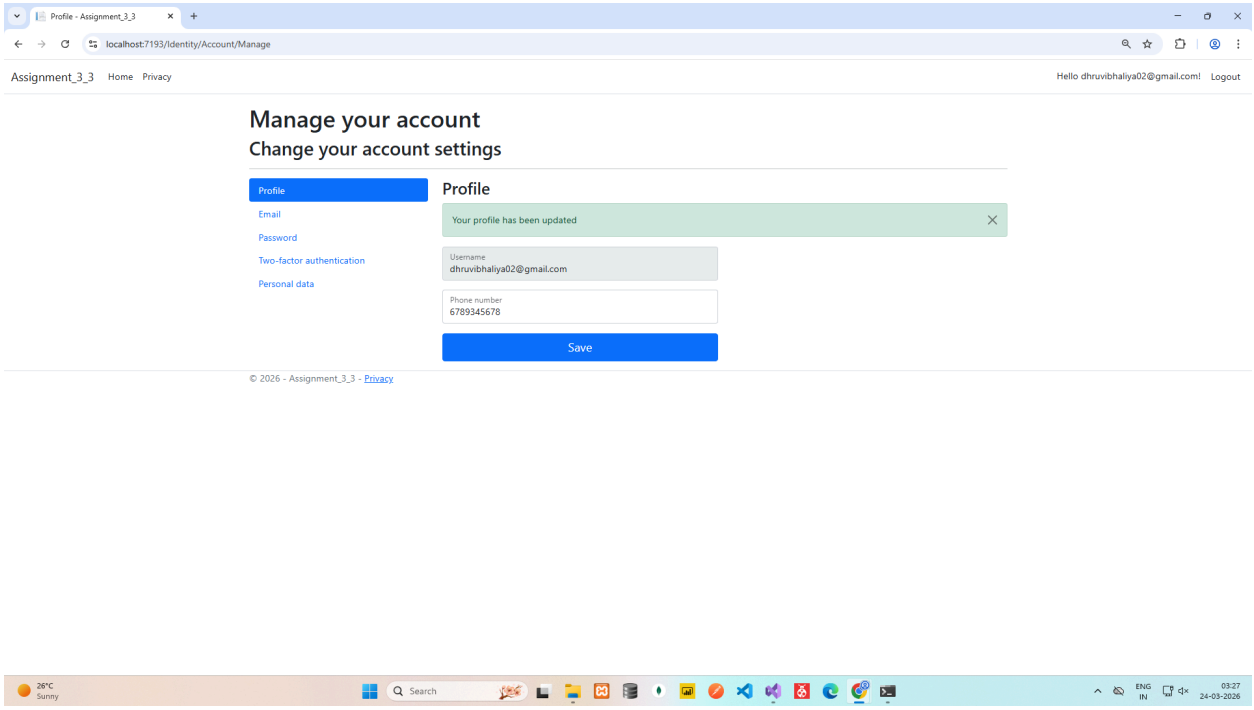
OUTPUT

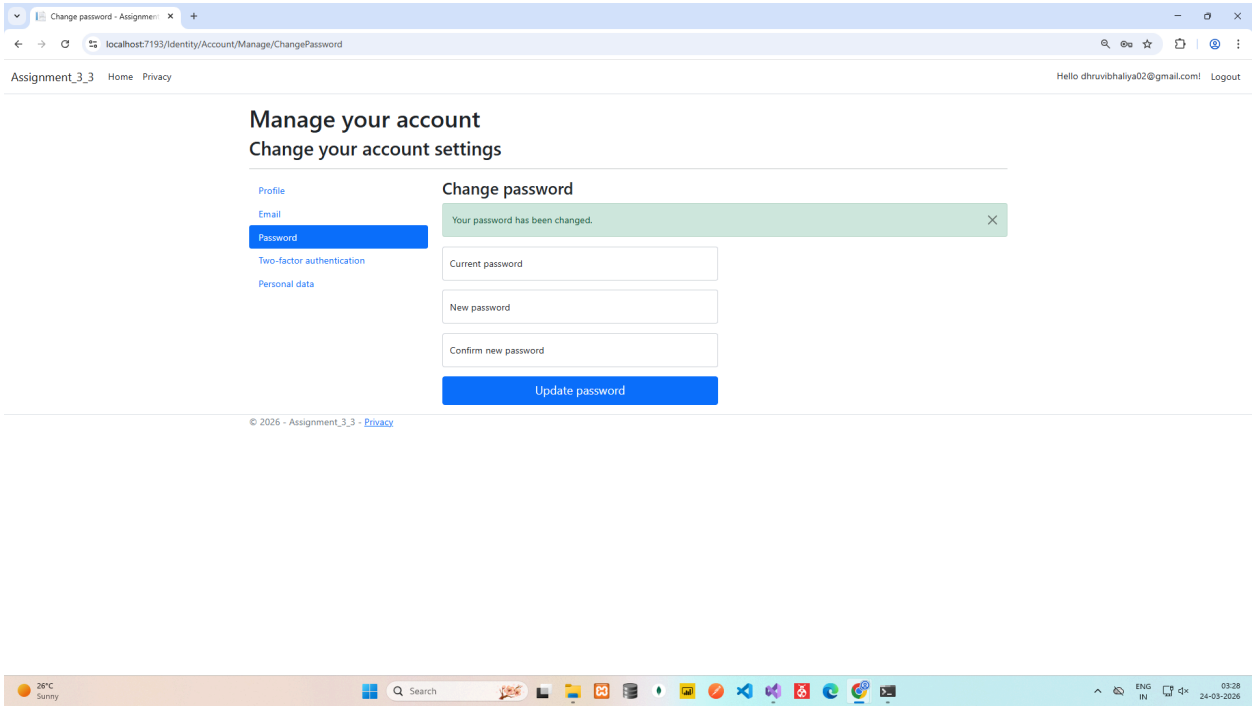

Register confirmation - Assignm
localhost:7193/Identity/Account/RegisterConfirmation?email=dhruvibhaliya01@gmail.com&returnUrl=%2F
Assignment_3_3 Home Privacy
Register Login
Register confirmation
This app does not currently have a real email sender registered, see these docs for how to configure a real email sender. Normally this would be emailed: Click here to confirm your account
© 2026 - Assignment_3_3 - Privacy

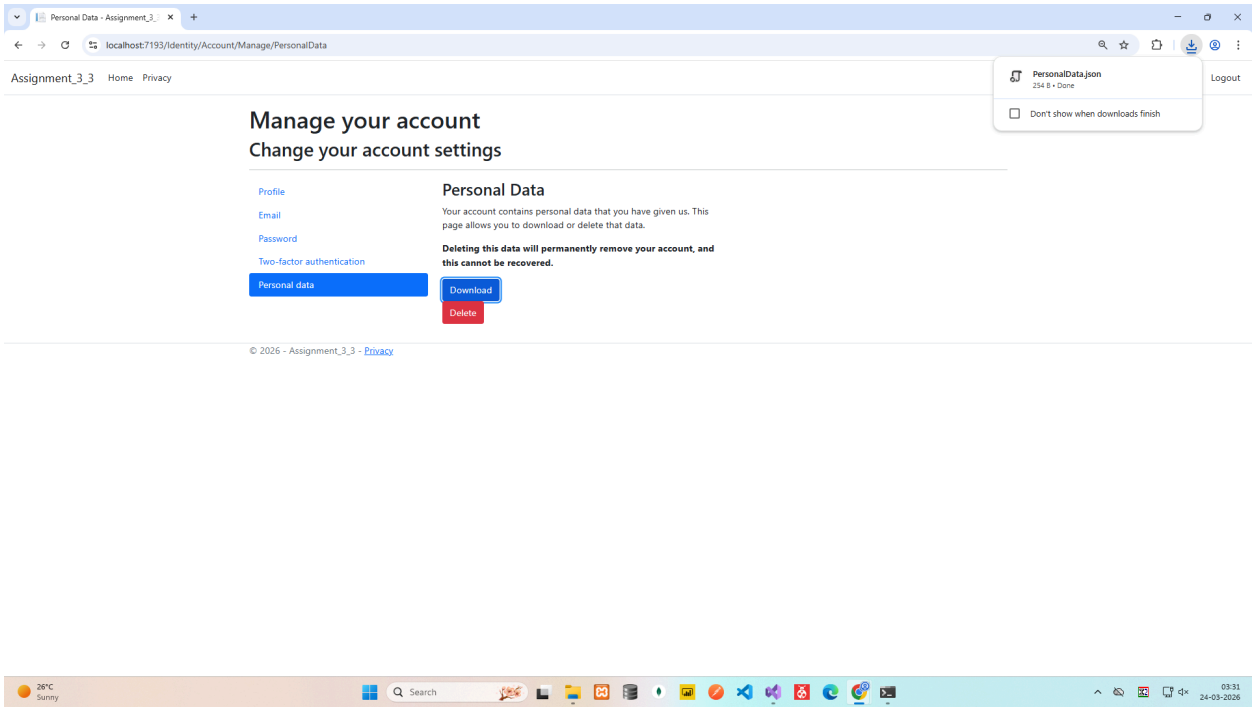
26°C
Sunny
Search
ENG
IN
03:24
24-03-2026

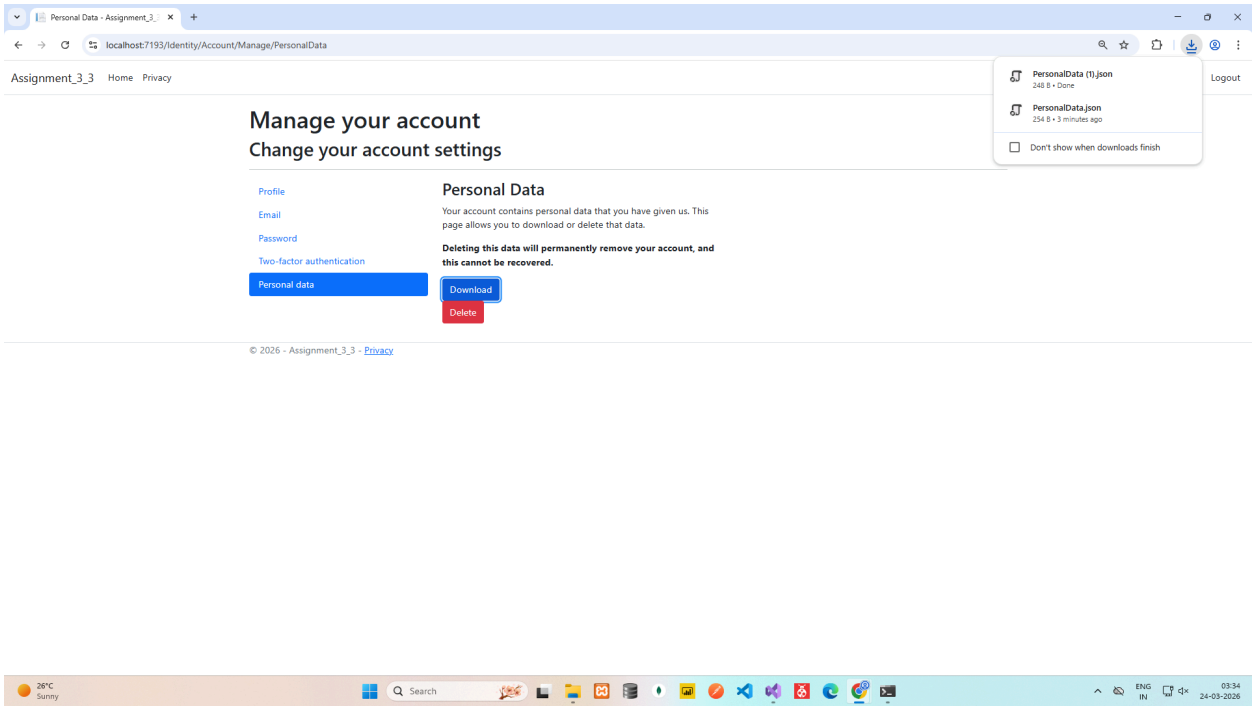

Confirm email - Assignment_3_
localhost:7193/Identity/Account/ConfirmEmail?userId=0203edb0-21f9-4fe0-afad-83d53a292c29&code=Q2ZESjhKRndERWw2WGJ4THYwM2IjV0N0REJPc05Vby9qdWJPWGxtU0V4WTdDS0MraEVQclcvbFgwWFk5dlNoc29XeUZaYUhZNEhDUGtFVWo5R1...
Assignment_3_3 Home Privacy
Register Login
Confirm email
Thank you for confirming your email.
© 2026 - Assignment_3_3 - Privacy

26°C
Sunny
Search
ENG
IN
03:26
24-03-2026


Profile - Assignment_3_3
localhost:7193/Identity/Account/Manage
Assignment_3_3
Home
Privacy
Hello dhruvibhaliya02@gmail.com!
Logout
Manage your account
Change your account settings
Profile
Email
Password
Two-factor authentication
Personal data
Profile
Your profile has been updated
Username
dhruvibhaliya02@gmail.com
Phone number
6789345678
Save
© 2026 - Assignment_3_3 - Privacy


Change password - Assignment
localhost:7193/Identity/Account/Manage/ChangePassword
Assignment_3_3
Home
Privacy
Hello dhruvibhaliya02@gmail.com!
Logout
Manage your account
Change your account settings
Profile
Email
Password
Two-factor authentication
Personal data
Change password
Your password has been changed.
Current password
New password
Confirm new password
Update password
© 2026 - Assignment_3_3 - Privacy


Personal Data - Assignment_3_
localhost:7193/Identity/Account/Manage/PersonalData
PersonalData.json
254 B • Done
Don't show when downloads finish
Assignment_3_3
Home
Privacy
Logout
Manage your account
Change your account settings
Profile
Email
Password
Two-factor authentication
Personal data
Personal Data
Your account contains personal data that you have given us. This page allows you to download or delete that data.
Deleting this data will permanently remove your account, and this cannot be recovered.
Download
Delete
© 2026 - Assignment_3_3 - Privacy
26°C
Sunny
Search
03:31
24-03-2026


Personal Data - Assignment_3_
localhost:7193/Identity/Account/Manage/PersonalData
PersonalData (1).json
248 B • Done
PersonalData.json
254 B • 3 minutes ago
Don't show when downloads finish
Assignment_3_3
Home
Privacy
Logout
Manage your account
Change your account settings
Profile
Email
Password
Two-factor authentication
Personal data
Personal Data
Your account contains personal data that you have given us. This page allows you to download or delete that data.
Deleting this data will permanently remove your account, and this cannot be recovered.
Download
Delete
© 2026 - Assignment_3_3 - Privacy
26°C
Sunny
Search
ENG
IN
03:34
24-03-2026